Perceptual Loss-Based Feedforward Neural Style Transfer

Emirkan Burak Yılmaz
emirkanyilmaz2019@gtu.edu.tr

July 9, 2025

Abstract

This study presents an implementation of a feedforward neural style transfer framework based on perceptual losses, enabling real-time stylization of images. The proposed method builds upon the architecture introduced by Johnson et al., in which a feedforward transformation network is trained using feature reconstruction losses derived from a fixed pretrained network. Multiple models are trained on the MS COCO dataset with different style targets and deployed via an interactive Gradio-based user interface. Qualitative results demonstrate competitive stylization performance while enabling efficient inference suitable for real-time applications.

1 Introduction

The objective of neural style transfer is to generate an image that preserves the semantic content of a photograph while adopting the visual appearance of an artwork. Traditional methods, such as the optimization-based approach introduced by Gatys et al. [1], rely on iterative updates during inference, which are computationally intensive, and not suitable for real-time applications. To address this limitation, feedforward networks trained with perceptual loss functions have been proposed, enabling style transfer to be performed in a single forward pass. In this study, a fast style transfer model based on the framework introduced by Johnson et al. [2] has been implemented with slight modifications and evaluated. The project source code, including training and inference scripts as well as trained models, is available at: `https://github.com/ebylmz/fast-neural-style-transfer`.

2 Related Work

Early approaches to style transfer involved patch-based synthesis and non-parametric methods. The seminal work of Gatys et al. [1] demonstrated that feature correlations computed from convolutional neural networks could effectively capture artistic style. However, their optimization-based inference process is slow, requiring hundreds of iterations. To overcome this limitation, Johnson et al. [2] introduced feedforward networks trained with perceptual losses derived from a pretrained classification network. Subsequent works have further extended this approach by introducing improvements such as arbitrary style transfer, adaptive instance normalization, and multi-style models.

3 Methodology

3.1 Datasets

The content dataset used in this study consists of approximately 82,000 images from the Microsoft COCO dataset, which contains a wide variety of natural scenes and objects, making it suitable for training style transfer models. Images were resized to 256×256 and center-cropped. Sample images are provided in Figure 1.

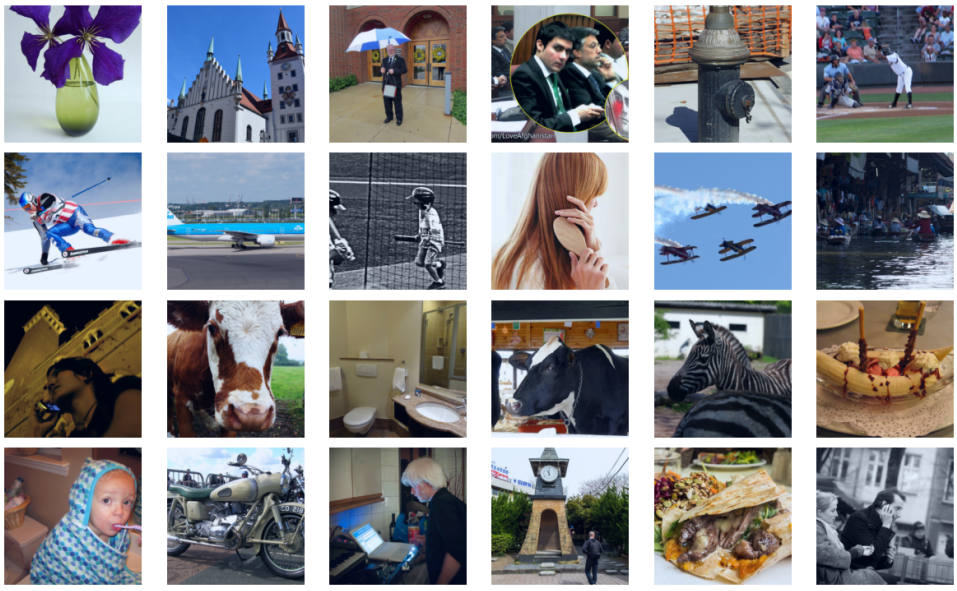

Figure 1: Sample images from Microsoft COCO dataset.

For style images, a small curated set of artworks was selected such as *Starry Night*, *La Muse*, and *Mosaic*, each representing a distinct style. A separate model was trained for each style. Figure 2 shows the selected style images.

Figure 2: Selected style images.

3.2 Model Architecture

The neural style transfer model is composed of two primary components: a transformation network and a loss network. The architecture proposed by Johnson et al. [2], illustrated in Figure 3, was adopted with minor modifications.

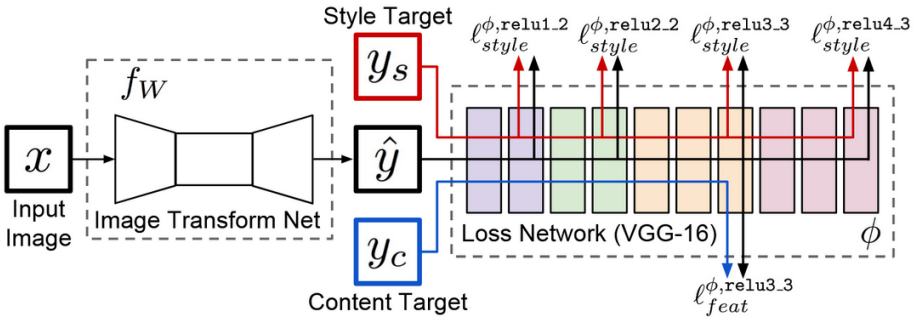

Figure 3: Model architecture based on Johnson et al. [2].

3.2.1 Transformation Network

The transformation network receives an input image and produces a stylized image of the same dimensions. It consists of convolutional layers with stride, residual blocks, and upsampling layers. Instance normalization is applied after each convolution layer. In contrast to the original implementation, pixel values were normalized to the range [0,1], and the output was clipped rather than passed through a scaled tanh activation. This modification led to more visually appealing results.

3.2.2 Loss Network

The loss network is a pretrained VGG-16 model trained on ImageNet. It remains fixed during training and is used solely to compute perceptual losses. Activations from intermediate layers are used to measure differences in content and style between the generated and target images.

3.3 Loss Functions

The total loss is a weighted sum of three terms: content loss, style loss, and total variation loss:

$$\mathscr{L} = \lambda_c \mathscr{L}_{\text{content}} + \lambda_s \mathscr{L}_{\text{style}} + \lambda_{tv} \mathscr{L}_{\text{tv}}$$

Content Loss: Content loss is computed as the squared difference between feature activations of the generated image $\hat{y}$ and the original content image y at layer ϕ_l:

$$\mathscr{L}_{\text{content}}(\hat{y}, y) = \frac{1}{2} \|\phi_l(\hat{y}) - \phi_l(y)\|_2^2$$

where layer l corresponds to `relu2_2` of the VGG network.

Style Loss: Style loss compares the style of the generated image to that of the target style image s using Gram matrices G_l:

$$\mathscr{L}_{\text{style}}(\hat{y}, s) = \sum_{l \in \mathscr{L}} \frac{1}{4N_l^2 M_l^2} \|G_l(\hat{y}) - G_l(s)\|_F^2$$

where $\mathscr{L}$ includes `relu1_2`, `relu2_2`, `relu3_3`, and `relu4_3`, and $G_l(x) = \phi_l(x)\phi_l(x)^T$.

Total Variation Loss: Total variation loss encourages spatial smoothness by penalizing high-frequency noise:

$$\mathscr{L}_{\text{tv}}(\hat{y}) = \sum_{i,j} \left((\hat{y}_{i,j+1} - \hat{y}_{i,j})^2 + (\hat{y}_{i+1,j} - \hat{y}_{i,j})^2 \right)$$

3.4 Training

The transformation network was trained using the Adam optimizer with a fixed learning rate of 10^{-3}. A batch size of 4 was used. Normalized images were fed through the network, and perceptual losses were computed with a fixed VGG-16 loss network. The loss was then backpropagated.

Each style model was trained with fixed content and total variation loss weights of $\lambda_c = 2.0$ and $\lambda_{tv} = 2.0$, while the style loss weight λ_s was individually tuned for each style in the range of 4×10^5 to 9×10^5 to balance stylization strength and content preservation. Training was performed for 1 epoch using the Microsoft COCO dataset on an NVIDIA A100 GPU in a Google Colab environment, requiring approximately an hour per model. In total, five separate models were trained, each corresponding to a unique style of image.

4 Experiments

5 models were trained on COCO images resized to 256×256 pixels. A separate transformation network was trained for each style image. Training curves and intermediate visual outputs are shown in Figures 4 through 8. These plots illustrate the evolution of the total loss and its components (content, style, and total variation) throughout the training process. In general, all loss components decrease steadily, although occasional spikes are observed in certain cases, such as in Figures 4 and 8. Minor fluctuations, particularly in the total variation loss, may be influenced by the complex spatial textures present in the target style images as well as the diverse content of the ImageNet dataset used for the pretrained loss network. Despite these variations, stable convergence was observed for all models within a single epoch of training on the COCO dataset.

4.1 Starry Night

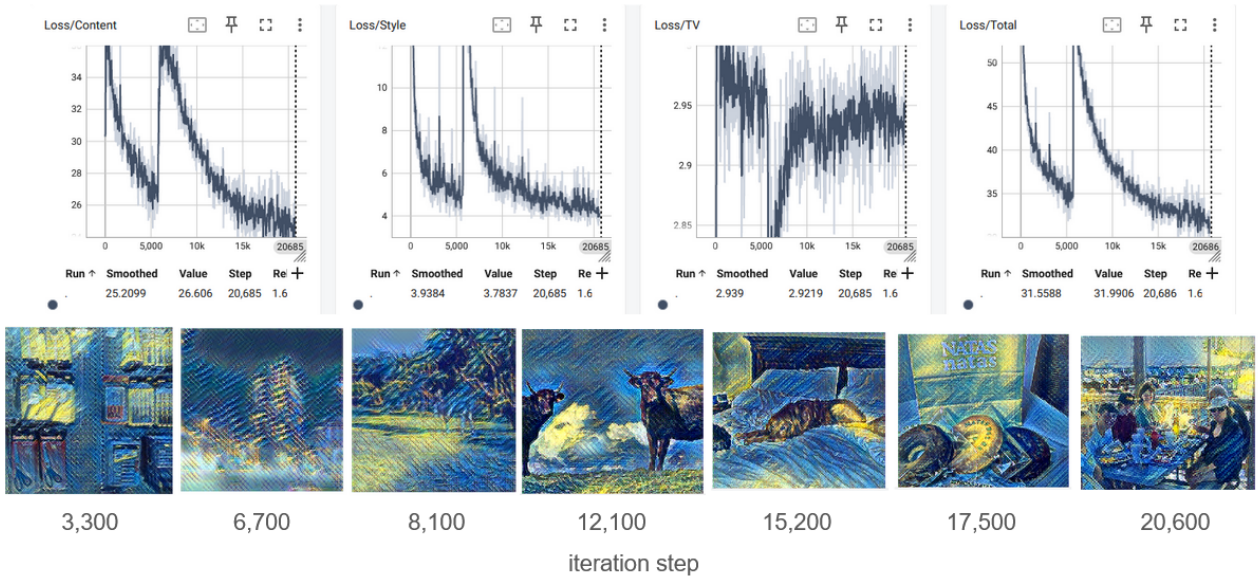

Figure 4: Training curves and intermediate results for Starry Night.

4.2 Mosaic

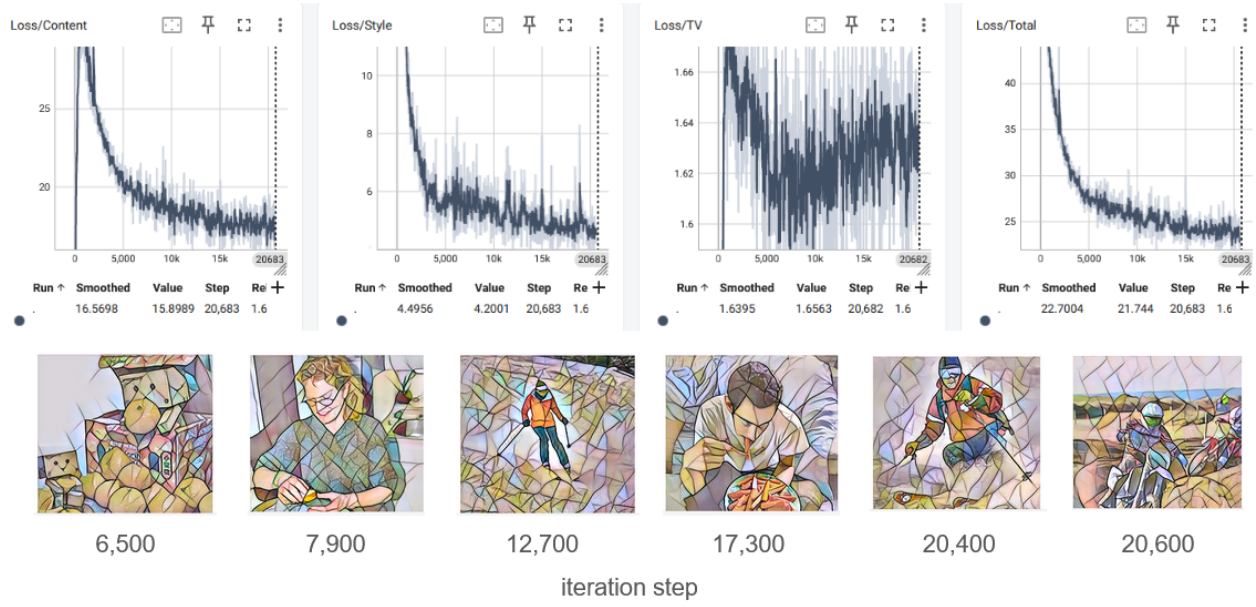

Figure 5: Training curves and intermediate results for Mosaic.

4.3 Crystal Grove

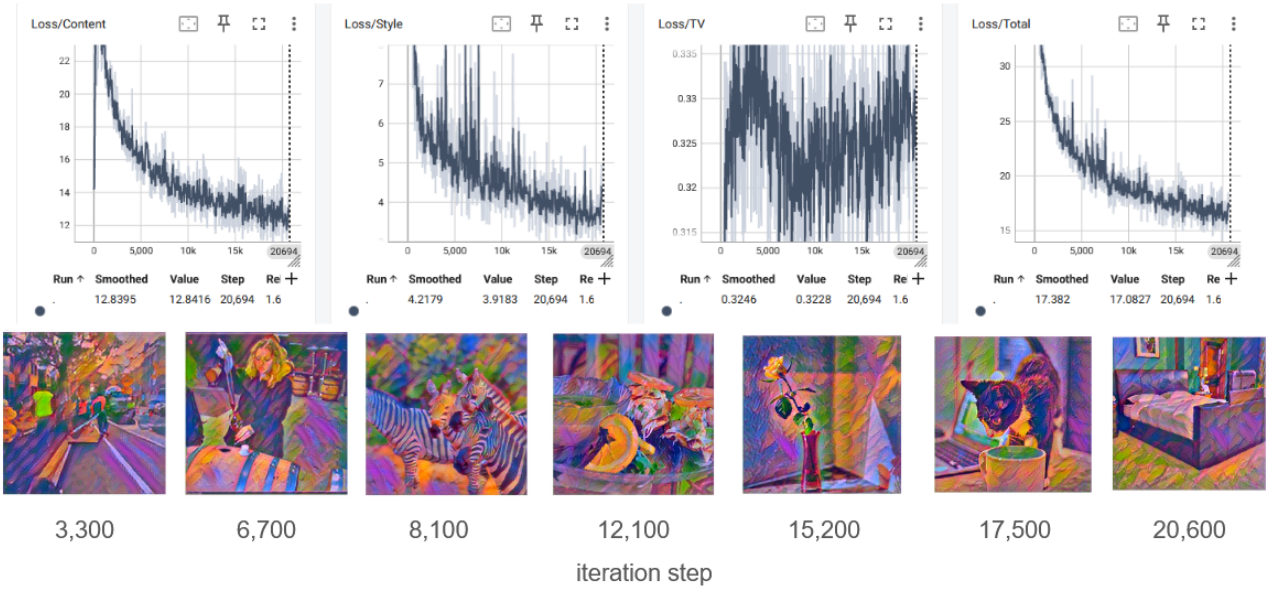

Figure 6: Training curves and intermediate results for Crystal Grove.

4.4 La Muse

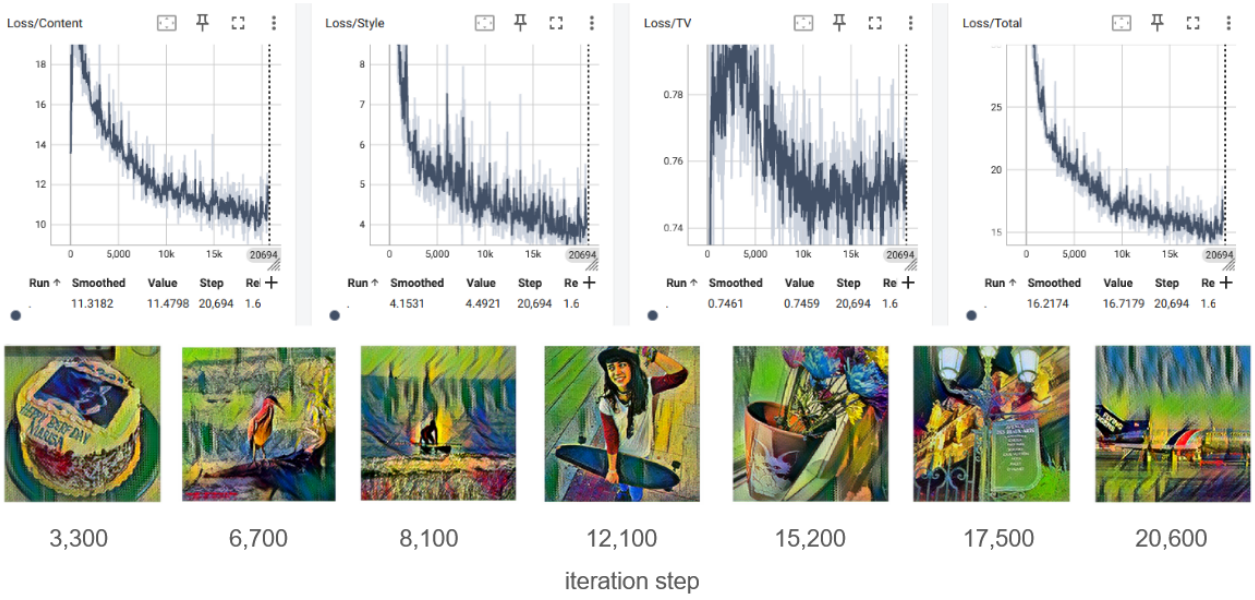

Figure 7: Training curves and intermediate results for La Muse.

4.5 Candy

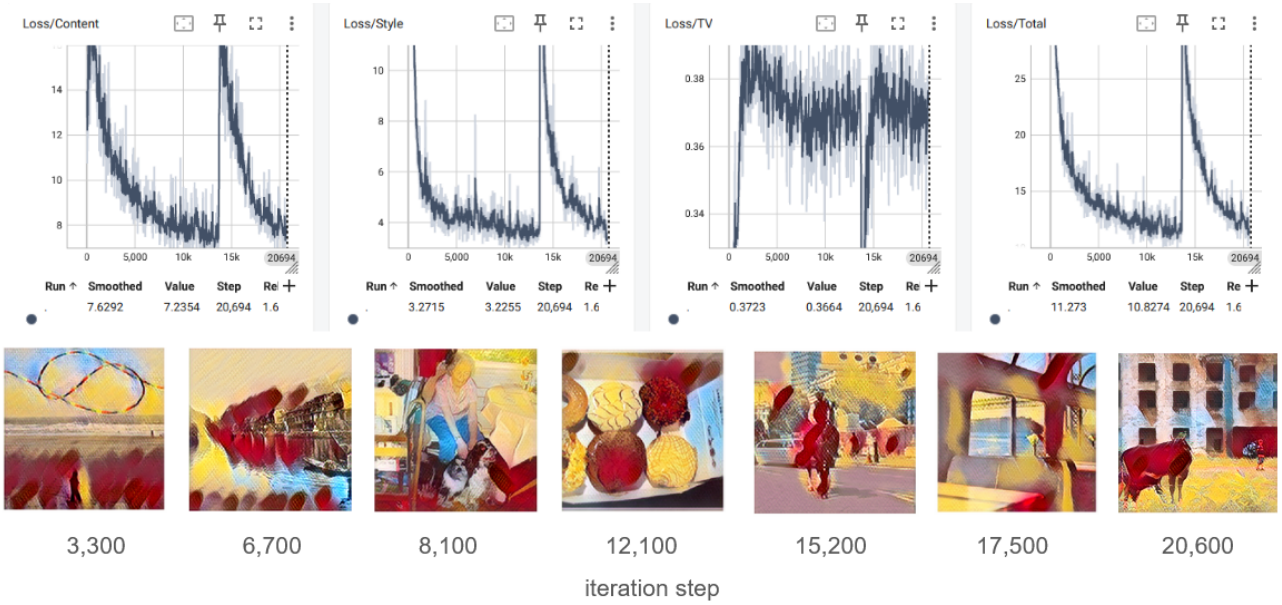

Figure 8: Training curves and intermediate results for Candy.

5 Results

A total of five models were trained. Example outputs are shown below.

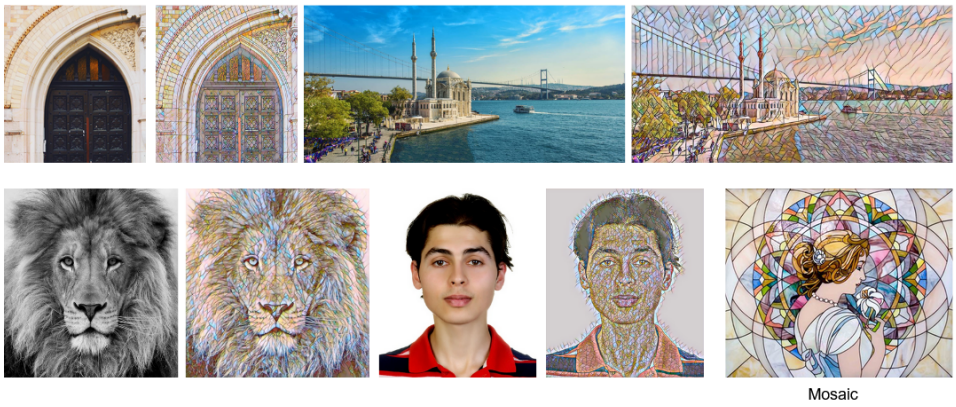

Figure 9: Mosaic style transfer results.

Figure 10: Starry Night style transfer results.

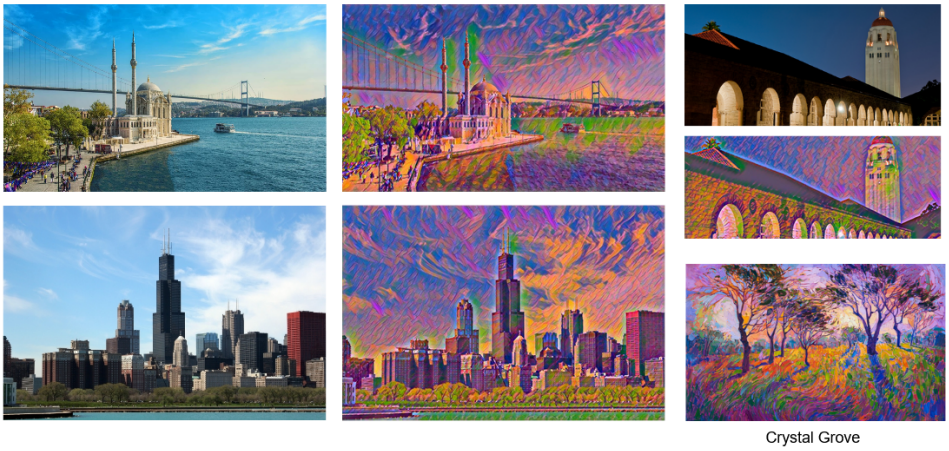

Figure 11: Crystal Grove style transfer results.

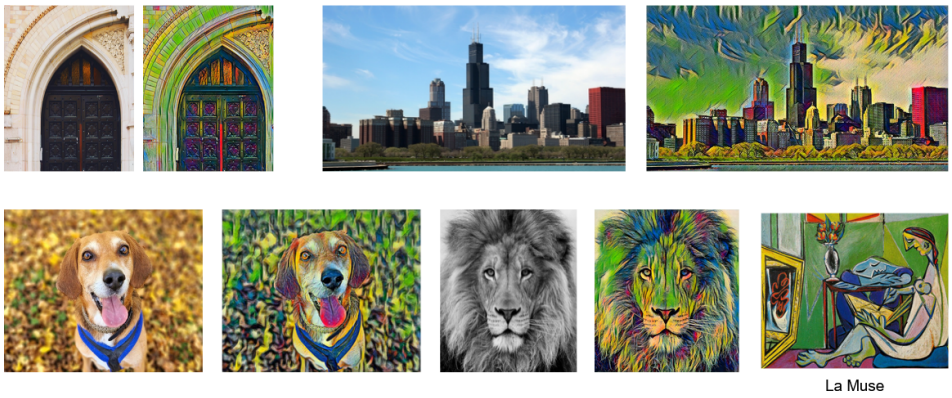

Figure 12: La Muse style transfer results.

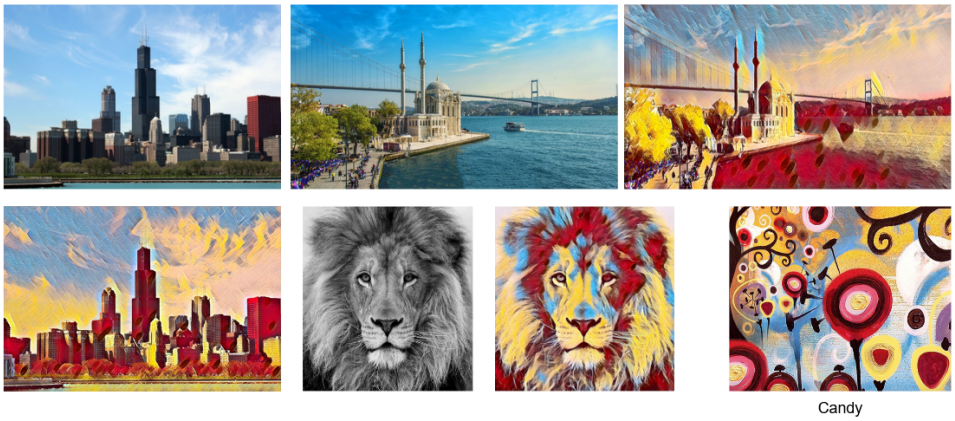

Figure 13: Candy style transfer results.

6 User Interface

An interactive user interface was developed using Gradio. The interface allows users to upload content images and select a style via clickable thumbnails. Upon clicking the stylize button, the generated image is returned in real-time.

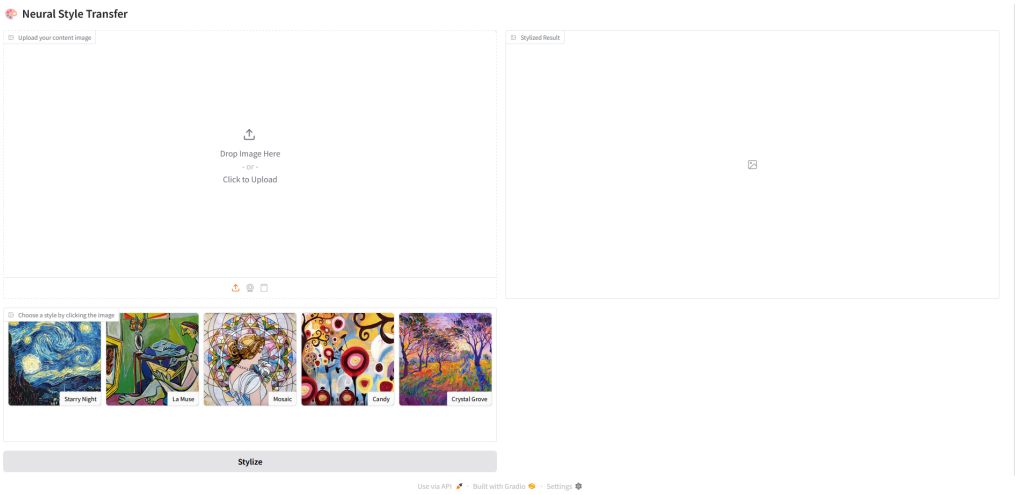

Figure 14: UI screen for content upload and style selection.

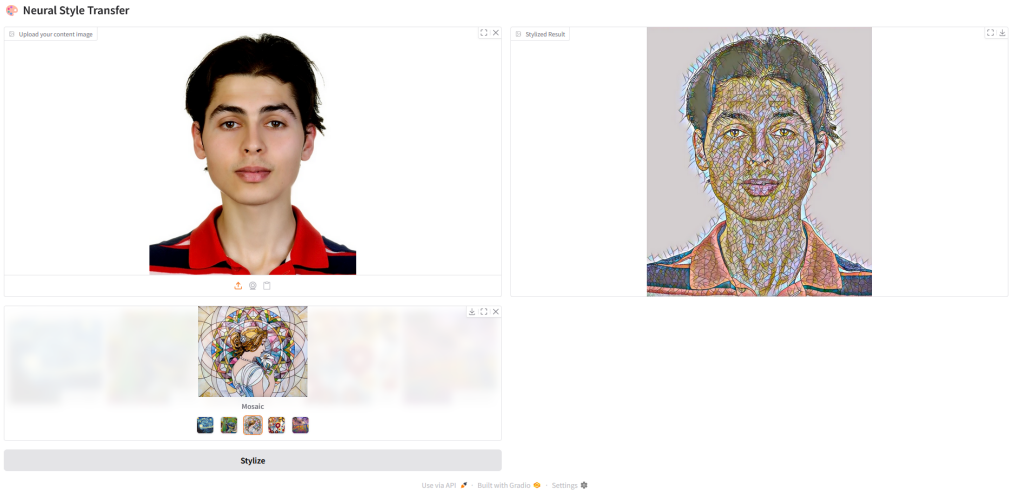

Figure 15: UI screen showing the stylized result.

7 Conclusion and Future Work

A feedforward neural network for real-time neural style transfer has been implemented and evaluated. The use of perceptual losses enables efficient stylization without iterative optimization. Future directions include multi-style models, arbitrary style transfer via adaptive normalization techniques, and integration of vision transformers or adversarial losses for improved generalization and style diversity.

References

[1] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2414–2423, 2016.

[2] J. Johnson, A. Alahi, and L. Fei-Fei, "Perceptual losses for real-time style transfer and super-resolution," in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II 14*, pp. 694–711, Springer, 2016.